

Key takeaways

Git

Các câu lệnh:

Câu lệnh	Công dụng
<code>git commit --amend</code> - Gõ i → vào chế độ Insert - Gõ esc để thoát Insert - Gõ ":wq" -> write and quit <code>git commit --amend -m"message"</code>	Thay đổi commit message
<code>git restore --staged <file></code>	Đưa từ vùng staging về working directory
<code>git reset HEAD~1 (undo 1 commit)</code>	Đưa từ 1 commit vùng repository về working directory (uncommit)
<code>git branch <branch_name></code>	Tạo branch
<code>git checkout <branch_name></code>	Di chuyển sang nhánh khác
<code>git checkout -b <branch_name></code>	Tạo và di chuyển sang branch khác

File .gitignore

.gitignore = GitIgnore = Bỏ qua

Dùng để bỏ qua các file không cần git theo dõi.

Ignore file

`<file_name>`

Ignore folder

`<folder-name>/`

Javascript basic

Conventions

- Snake_case: snake_case_now_now
- kebab-case: kebab-case-now-now
- camelCase: camelCaseNowNow
- PascalCase: PascalCaseNowNow

Formatted console.log

```
console.log('Toi la Nga');  
console.log("Toi la Phong");  
console.log(`${variable_name}`)  
let name = "Nga";  
console.log(`Toi la ${name}`);  
console.log("Toi ten la" + name + " ")
```

Object

OOP và Object

Có thể bạn có từng nghe về khái niệm OOP (Object-Oriented Programming), hay lập trình hướng đối tượng. Object chính là trái tim của OOP.

Kiểu dữ liệu đối tượng, hay Object, là kiểu dữ liệu lưu trữ một **tập hợp giá trị** dưới dạng **key-value**. Object sẽ lưu dữ liệu theo thành từng "đối tượng", khiến dữ liệu trở nên gọn gàng và dễ quản lý hơn.

Cú pháp

Để sử dụng Object, ta dùng cú pháp khai báo với let hoặc const:

```
let/const <object_name> = {  
  'key1': value1,  
  'key2': value2,  
  ...  
  'keyN': valueN
```

```
}
```

Trong đó 'key' là các 'khoá' giúp chúng ta lấy ra các dữ liệu tương ứng sau này, value là các giá trị tương ứng với một 'khoá'. Các key cũng có thể gọi là các property (thuộc tính) của object.

Kiểu dữ liệu của key luôn là string.

Kiểu dữ liệu của value có thể là bất cứ kiểu dữ liệu nào đã học: string, number, boolean, null, object, array,...

Dùng object và không dùng object

Ví dụ với việc quản lý thông tin sinh viên, giả sử bạn cần quản lý 3 sinh viên với các thông tin: mã sinh viên, tên sinh viên, tuổi.

Không dùng object

Khi không dùng object, ta cần khai báo các biến riêng biệt để lưu trữ dữ liệu cho các thông tin của sinh viên

```
const sv1ID = 1;  
const sv1Name = "Alex";  
const sv1Age = 20;
```

```
const sv2ID = 2;  
const sv2Name = "Nagi";  
const sv2Age = 18;
```

```
const sv3ID = 3;  
const sv3Name = "Lua";  
const sv3Age = 19;
```

Khi cần in ra thông tin sinh viên, ta cũng dùng các biến riêng lẻ để thực hiện lấy giá trị ra

```
console.log(`- Thông tin SV1: ${sv1ID}, ${sv1Name}, ${sv1Age}`);  
console.log(`- Thông tin SV2: ${sv2ID}, ${sv2Name}, ${sv2Age}`);  
console.log(`- Thông tin SV3: ${sv3ID}, ${sv3Name}, ${sv3Age}`);
```

Việc quản lý các biến rất dễ gây nhầm lẫn. Ví dụ, bạn có thể gõ nhầm biến `sv3Name` cho sinh viên 2.

Dùng object

Dùng object, ta khai báo đối tượng sinh viên như sau:

```
const sv1 = {
```

```
"ID": 1,  
"name": "Alex",  
"age": 20  
};
```

Bây giờ, chúng ta chỉ có 1 hằng số `sv1` duy nhất lưu trữ thông tin của sinh viên. Khi truy cập các thuộc tính của sinh viên 1, ta có thể truy cập thông qua việc dùng dấu `.`:

```
console.log(sv1.ID);
```

hoặc dùng ngoặc vuông [`"<key>"`]:

```
console.log(sv1["name"]);
```

Object lồng nhau

Một object có thể chứa một object khác. Ví dụ dưới đây mô tả object `student` chứa một object `address`:

```
const student = {  
  id: 1,  
  name: "Alex",  
  address: {  
    province: "Ha Noi",  
    isCapital: true,  
    country: "Viet Nam"  
  }  
}
```

Thêm thuộc tính vào object

Để thêm thuộc tính mới vào object, chúng ta chỉ cần dùng dấu `.` hoặc ngoặc vuông `[]` để định nghĩa thuộc tính mới. Ví dụ:

```
let bike = {  
  make: 'Yamaha',  
  model: 'YZF-R3'  
};
```

```
bike.color = "Blue";
```

```
bike["price new"] = 100;
```

```
console.log(bike);
```

Kết quả:

```
{make: 'Yamaha', model: 'YZF-R3', color: 'Blue', 'price new' :  
100}
```

Xóa thuộc tính của object

Để xóa thuộc tính của object, chúng ta dùng hàm `delete`:

```
let employee = {  
  name: 'Le Van C',  
  age: 30,  
  department: 'HR'  
};  
delete employee.age;  
console.log(employee);
```

// Kết quả:

```
{name: 'Le Van C', department: 'HR'}
```

Tips

1. Các thuộc tính có thể khai báo nằm trên cùng một dòng, có thể dùng nháy đơn hoặc nháy kép để khai báo thuộc tính.

```
const student = { 'id': 10 };  
const student2 = { "id": 20 };
```

2. Đối với các thuộc tính không có dấu cách, có thể bỏ dấu nháy đơn và nháy kép.

```
const student = { id: 10 };  
const student2 = { id: 20 };
```

3. Thông thường, chúng ta hạn chế đặt tên thuộc tính có dấu cách, để việc khai báo và truy cập thuộc tính dễ dàng hơn.

Không nên: đặt tên thuộc tính chứa dấu cách

```
const student = { 'my name': "Alex" };
```

Nên: đặt tên thuộc tính sử dụng camelCase, không chứa dấu cách.

```
const student = { myName: "Alex" };
```

4. Ta không thể thay đổi giá trị của hằng số. Tuy nhiên, nếu hằng số này là một Object, ta có thể thay đổi giá trị các **thuộc tính** của chúng.

Không thể làm thế này:

```
const student = { id: 10, name: "Alex" };  
student = { id: 11, name: "Nagi" };  
//      ^
```

// TypeError: Assignment to constant variable.

Có thể làm thế này:

```
const student = { id: 10, name: "Alex" };  
student.id = 11;  
student.name = "Nagi";
```

Logical operator

Là các toán tử dùng để kết hợp các giá trị với nhau.

Toán tử	Ví dụ	Kết quả	Ý nghĩa
&&	true && true true && false false && false	true false false	Phép và, trả về true nếu cả 2 mệnh đề đều đúng
	false true true true false false	true true false	Phép hoặc, trả về true nếu một trong hai mệnh đề đúng
!	!true !false	false true	Phép nghịch đảo, trả về giá trị ngược lại của biểu thức

Array: mảng

Định nghĩa

Mảng (hay array) là một đối tượng trong Javascript, giúp lưu trữ một tập hợp các giá trị, lưu trữ dưới một tên biến và có thể sử dụng các thao tác trên mảng.

Một mảng có thể chứa **nhiều loại dữ liệu khác nhau**.

Khai báo mảng

Để khai báo mảng, ta sử dụng cú pháp:

```
let/const/var <tên_mảng> = [<danh sách các giá trị, cách nhau bởi dấu phẩy ">,">
```

Ví dụ:

```
let numberArr = [1, 20.5, -300, 4];
const strArr = ["Playwright", "Việt", "Nam"];
var mixedArr = ["Playwright", 10, true, null, {id: 1, name: "Alex"}];
```

```
console.log(numberArr);
console.log(strArr);
console.log(mixedArr);
```

```
// Kết quả:
// [ 1, 20.5, -300, 4 ]
// [ 'Playwright', 'Việt', 'Nam' ]
// [ 'Playwright', 10, true, null, { id: 1, name: 'Alex' } ]
```

Giải thích:

- Mảng `numberArr` chứa các dữ liệu kiểu số.
- Mảng `strArr` chứa các dữ liệu kiểu chuỗi.
- Mảng `mixedArr` chứa các dữ liệu kiểu hỗn hợp: chuỗi, số, boolean, object.

Thao tác trên mảng

Lấy độ dài của mảng

Để lấy độ dài của mảng, ta dùng thuộc tính length của array:

```
const arr = [1, 2];  
console.log(arr.length);  
// Kết quả  
// 2
```

Thường ứng dụng trong việc lặp qua các phần tử trong mảng

```
const arr = [20, 50, 30, 40];  
  
for (let i = 0; i < arr.length; i++) {  
    console.log(arr[i]);  
}  
// Kết quả  
// 20  
// 50  
// 30  
// 40
```

Truy xuất phần tử của mảng

Để truy xuất phần tử trong mảng, ta dùng cú pháp `arr[<index>]`, trong đó index là số thứ tự, tính từ 0.

```
const udemy = [20, 50, 30, 40];  
  
console.log(udemy[0]);  
console.log(udemy[1]);  
console.log(udemy[2]);  
console.log(udemy[3]);  
// Kết quả  
// 20
```



```
// 50
```

```
// 30
```

```
// 40
```

Giải thích:

- Mảng có 4 phần tử: 20, 50, 30, 40
 - Phần tử 20: index = 0
 - Phần tử 50: index = 1
 - Phần tử 30: index = 2
 - Phần tử 40: index = 3
- Khi ta sử dụng: `a[3]` tức là đang lấy phần tử có vị trí số 3.

Thêm phần tử vào mảng

Thêm vào đầu mảng

Sử dụng hàm `unshift()`:

```
const arr = [20, 50];
```

```
arr.unshift(10);
```

```
console.log(arr);
```

```
// Kết quả
```

```
// [ 10, 20, 50 ]
```

Thêm vào cuối mảng

Sử dụng hàm `push()`:

```
const arr = [20, 50];
```

```
arr.push(10);
```

```
console.log(arr);
```

```
// Kết quả
```

```
// [ 20, 50, 10 ]
```

Xóa phần tử khỏi mảng

Xóa phần tử đầu của mảng

Sử dụng hàm `shift()`

```
const arr = [20, 50, 10];  
const deleted = arr.shift();
```

```
console.log(arr);  
console.log(deleted);  
// Kết quả  
// [ 50, 10 ]  
// 20
```

Giải thích:

- Hàm `shift()` xóa phần tử đầu tiên của mảng và trả về.

Xóa phần tử cuối của mảng

Sử dụng hàm `pop()` để xóa phần tử cuối cùng của mảng

```
const arr = [20, 50, 10];  
const deleted = arr.pop();
```

```
console.log(arr);  
console.log(deleted);  
// Kết quả  
// [ 20, 50 ]  
// 10
```

Giải thích:

- Hàm `pop()` lấy giá trị cuối cùng của phần tử ra khỏi mảng và trả về.

Tips

- Mặc dù mảng có thể chứa nhiều kiểu dữ liệu khác nhau, nhưng trong thực tế, chúng ta thường chỉ sử dụng một loại dữ liệu duy nhất cho một mảng
Không nên: khai báo mix các kiểu dữ liệu trong cùng một mảng

```
var mixedArr = ["Playwright", 10, true, null, {id: 1, name: "Alex"}];
```

Nên: tách kiểu dữ liệu tương ứng thành từng mảng.

```
let numberArr = [1, 20.5, -300, 4];  
const strArr = ["Playwright", "Việt", "Nam"];
```

Function: Hàm

Khái niệm hàm

Hàm là một khối code. Hàm thường được dùng để viết các đoạn logic lặp đi lặp lại.

Khai báo hàm

Cú pháp

```
function <tên_hàm(<danh_sách_tham_số>) {  
    // Code của hàm  
}
```

Trong đó:

- **function**: từ khoá khai báo hàm
- **<tên_hàm>**: tên của hàm.
- **<danh sách tham số>**: các tham số của hàm, cách nhau bởi dấu phẩy “,”
- **Code của hàm**: đoạn logic thực thi trong hàm.

Ví dụ với hàm đơn giản

```
function hello() {  
    console.log("Hello Playwright Viet Nam");  
}
```

Gọi hàm

Hàm không thể tự chạy được. Muốn chạy, bạn cần “gọi hàm”.

Hàm không có tham số

Đối với hàm không có tham số, ta chỉ cần “gọi hàm” bằng cách viết tên hàm và thêm cặp ngoặc tròn phía sau:

```
hello();
```

Hàm có tham số

Tham số giúp hàm trở nên linh động hơn. Tham số nằm trong cặp ngoặc tròn ().

Ví dụ

```
function describePerson(name, age, city) {  
    console.log(name + " is " + age + " years old and lives in " +  
city + ".");  
}
```

```
describePerson("Bob", 30, "New York");  
// Kết quả: Bob is 30 years old and lives in New York.
```

Hàm có giá trị trả về

Hàm có giá trị trả về là hàm trả về một giá trị sau khi hoàn thành việc thực thi. Giá trị trả về được chỉ định bằng từ khóa `return`.

Ví dụ

```
function add(a, b) { // a và b là tham số  
    return a + b; // Trả về tổng của a và b  
}
```

```
let sum = add(5, 3);
```